VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

PROJECT REVIEW 1

Memory & Swap Optimizer Agent

A natural-language to command-line agent for Linux memory management using the Model Context Protocol

Submitted by

S.No	Name	Register Number
1	Ashish Kumar	24BKT0059
2	Kumar Hrishikesh Ranjan	24BDS0221

Faculty Guide: Naveenkumar J

October 2026

Table of Contents

Chapter 1: Introduction **3**

1.1 Project Overview 3

1.2 Problem Statement 3

1.3 Objectives 4

1.4 Scope and Limitations 4

Chapter 2: Literature Review & Theoretical Background **6**

2.1 Large Language Models (LLMs) in Code Generation 6

2.2 Introduction to Model Context Protocol (MCP) 6

2.3 Existing Solutions 8

References **10**

Chapter 1: Introduction

1.1 Project Overview

The command line has been the main way of controlling Unix-like systems for more than fifty years. Early users worked through teletype terminals and the original Bourne shell, which gave them a small set of composable programs connected by pipes. Over time the shell grew into a full scripting environment (Bash, Zsh, and later PowerShell on Windows), and the number of utilities, options and configuration files that an administrator is expected to know grew with it. Graphical tools made everyday tasks easier, but servers, containers, cloud machines and most developer workstations are still administered through a terminal because it is scriptable, fast and works over a remote connection.

In the last few years Large Language Models (LLMs) have changed how people can talk to such systems. Instead of recalling the exact syntax of a command, a user can state what they want in ordinary English, and a model can propose the commands that match that intent. This is the idea behind natural-language-to-CLI translation, an area first studied with sequence-to-sequence models on datasets such as NL2Bash and now much more practical with instruction-tuned models that can call external tools.

This project builds one specialised agent in a larger set of Linux administration agents: the **Memory & Swap Optimizer Agent**. A typical request it should handle is: *"My computer is lagging. Free up some RAM cache and check for memory leaks."* The agent inspects the current memory state, decides whether the problem is page-cache pressure, heavy swapping or a single misbehaving process, and then carries out a limited and safe set of actions. It does this by exposing memory-related utilities as tools through the Model Context Protocol (MCP), so the model works from real measurements and not from guesses.

1.2 Problem Statement

Syntactical barriers and learning curve. Diagnosing a slow machine needs several tools and some background knowledge. A user has to know that `free -h` gives a summary, that `vmstat` shows swap-in and swap-out activity, that `smem` reports proportional set size, and that the real indicator of available memory is the `MemAvailable` field in `/proc/meminfo` and not the "free" column. Many users also confuse cached memory with used memory and become worried when Linux uses most of the RAM for caching, which is normal behaviour. The shells themselves (Bash and PowerShell) add to the difficulty with their different quoting rules, flags and output formats.

Risk of human error. Memory management involves privileged interfaces. Writing to the wrong file under `/proc/sys`, running `swapoff` when there is not enough free RAM to take the swapped pages back, or killing the wrong process can make the machine unresponsive, trigger the kernel's out-of-memory (OOM) killer, or lose unsaved work.

Copying commands from forums without understanding them is a common source of such mistakes.

The problem addressed here is therefore: *how can a non-expert user manage memory and swap on a Linux machine in plain language, while the system keeps the risk of wrong or harmful commands low?*

1.3 Objectives

Primary objective. To build a lightweight, secure natural-language-to-CLI translation engine for memory and swap management. Lightweight is important for this project in particular, because a tool that is meant to relieve memory pressure must not itself use a large amount of memory. The following specific objectives were set:

- To translate a user request into one or more validated Linux commands or tool calls, and to explain in simple words what was done and why.
- To integrate context-aware tools using the Model Context Protocol (MCP), so that the model can first read live values such as MemAvailable, swap usage and the top memory-consuming processes before it decides on an action.
- To implement the main functions of the agent: cache release by writing to `/proc/sys/vm/drop_caches`, swap inspection and tuning, and detection of suspected memory leaks by sampling process memory over time.
- To protect the system through an allow-list of commands, argument validation, a dry-run mode and explicit user confirmation for every action that changes system state.
- To log each request, tool call and result so that the actions of the agent can be audited afterwards.

1.4 Scope and Limitations

Supported operating systems and terminal environments. The agent is designed for Linux distributions based on Debian and Ubuntu, using Bash as the terminal environment. It depends on the `/proc` and `/sys` virtual filesystems and on standard utilities such as `free`, `vmstat`, `ps`, `swapon` and `sysctl`. Support for Windows (PowerShell) and macOS is outside the scope of this review. Containers and virtual machines are supported only to the extent that the kernel interfaces are visible inside them.

Extent of administrative privileges granted to the AI model. The model is never given an open root shell. It can only call the specific tools listed by the MCP server. Read-only tools run without prompting. Tools that modify the system run only after the user confirms, and the server uses the minimum privilege needed for that single call. Arbitrary command execution, package installation and process termination are not exposed in this version.

Known limitations. Dropping the page cache does not fix a real memory leak. It only releases clean cached data, and the kernel normally reclaims this memory on its own when applications need it. After the drop, files must be read from disk again, so performance may be slightly worse for a short time. The agent therefore treats the cache drop as a diagnostic or last-step option and explains this to the user. Leak detection is based on observed growth of resident memory and can give false positives for programs that legitimately grow, such as browsers or databases. Finally, the quality of the translation depends on the chosen LLM, which is why validation is done outside the model.

Chapter 2: Literature Review & Theoretical Background

2.1 Large Language Models (LLMs) in Code Generation

Capabilities of modern LLMs. Transformer-based language models trained on large text and code corpora can produce working code in many languages. Brown et al. showed that large models can follow a task from a few examples in the prompt, and Chen et al. evaluated a code-specialised model on the HumanEval benchmark, where it solved a significant share of programming problems. For the command line, Lin et al. introduced the NL2Bash corpus, which pairs English descriptions with Bash one-liners and made the translation problem measurable. More recent models also support structured outputs and function calling, so they can return a JSON object that names a tool and its arguments instead of free text. Work such as ReAct and Toolformer showed that interleaving reasoning with tool use lets a model gather information before it answers, which is the pattern this project relies on.

Challenges with hallucination in system-level execution. A language model produces the most likely continuation of its input and not a verified fact. Ji et al. survey this problem under the name hallucination. In a chat answer, a wrong statement is only an annoyance. In system administration, an invented flag, a wrong path or an over-broad command can damage the system. Typical failure cases include confusing `drop_caches` values 1, 2 and 3, assuming that clearing the cache always frees memory for good, or disabling swap on a machine that has no spare RAM. The design response used in this project is to keep the model away from the shell: it can only select from typed tools, its arguments are checked against a schema, and dangerous actions require confirmation.

2.2 Introduction to Model Context Protocol (MCP)

Architecture of MCP. The Model Context Protocol is an open standard, introduced by Anthropic in 2024, for connecting AI applications to external data and tools. It is based on JSON-RPC 2.0 messages and defines three roles. The *host* is the AI application that the user interacts with, and it contains an *MCP client* for each connection. An *MCP server* is a separate program that exposes capabilities. These capabilities are of three kinds: *tools* (functions the model can call, described with a JSON schema for the inputs), *resources* (readable data) and *prompts* (reusable templates). A local server normally communicates with the client over standard input and output (stdio), while remote servers use HTTP-based transports. During start-up the client asks the server for its tool list, and the model receives each tool's name, description and input schema as context.

In this project the MCP server wraps a small number of memory tools, listed in Table 2.1. Each tool returns structured output (for example, JSON with values in kilobytes) so that the model does not have to parse free-form terminal text.

Tool name	Type	Purpose	Underlying source
get_memory_stats	Read-only	Total, available, cached memory and swap usage	/proc/meminfo
top_memory_processes	Read-only	Largest processes by resident and proportional memory	ps, smem, /proc/[pid]/status
sample_process_growth	Read-only	Tracks memory of a process over time to detect leaks	/proc/[pid]/status
get_swap_info	Read-only	Swap devices, usage, swappiness and swap-in/out rates	swapon --show, vmstat, sysctl
drop_caches	Write (confirm)	Flushes dirty data, then frees page cache and/or dentries and inodes	sync, /proc/sys/vm/drop_caches
set_swappiness	Write (confirm)	Changes the kernel's tendency to swap	sysctl vm.swappiness

Table 2.1: Tools exposed by the Memory & Swap Optimizer MCP server

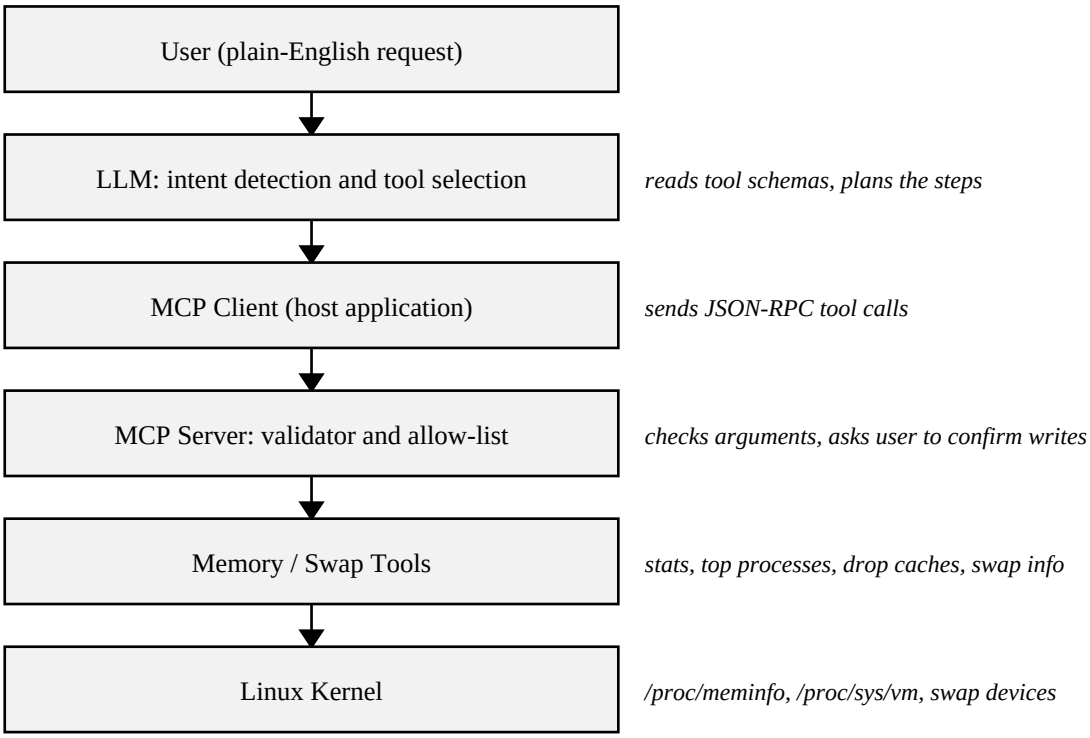

Figure 2.1: Proposed architecture of the Memory & Swap Optimizer Agent

Why MCP is preferred over rigid API tooling. Before MCP, each application had to build its own integration for every tool and every model, which resulted in many one-off

connectors. With MCP, a tool is described once and can then be used by any compatible client. The tool list is discovered at run time, so adding a new tool to the server does not require changes to the client. For local system access there are two more advantages. First, a stdio server runs on the same machine, so memory statistics and process names never have to be sent to a third-party service by the tool layer. Second, the server is the single place where permissions, validation and logging are applied, which is easier to audit than commands scattered through prompts.

Linux memory concepts used by the agent. The agent's decisions rest on a few kernel concepts. The *page cache* keeps recently used file data in RAM; it is counted as used by simple tools but is reclaimable, so the correct figure for memory that applications can still obtain is `MemAvailable`. *Dentries and inodes* are cached filesystem metadata. *Swap* is disk space used to hold anonymous pages that are not recently needed, and the `vm.swappiness` parameter controls how willing the kernel is to swap compared with reclaiming file cache. Writing 1, 2 or 3 to `/proc/sys/vm/drop_caches` frees the page cache, the dentries and inodes, or both; the kernel documentation advises running `sync` first because only clean pages are dropped, and describes the interface as a debugging aid and not a normal tuning method. Sustained non-zero swap-in and swap-out rates in `vmstat` are a stronger sign of real memory pressure than a high swap-used figure alone. The OOM killer is the last-resort mechanism that ends a process when memory is exhausted, and the agent's aim is to detect pressure early enough that it is never reached.

2.3 Existing Solutions

Comparative study. Several tools already bring AI to the terminal. **GitHub Copilot CLI** began as a command-suggestion and explanation feature (`gh copilot suggest` and `explain`) and has since moved toward agentic use with user approval of actions; it is a general-purpose assistant and does not target memory management. **Warp AI** is built into the Warp terminal and can generate commands and assist with errors, but it is tied to that terminal application and a cloud service. Open-source alternatives such as **ShellGPT** and **Open Interpreter** let a user connect different models and run generated commands locally, but they usually give the model broad shell access with little structure or protection, which is the risk this project wants to reduce.

Outside the AI category, memory management on Linux is handled by conventional tools. Monitors such as **top**, **htop**, **glances** and **Netdata** show the state of the system but leave interpretation and action to the user. Daemons such as **earlyoom** and **systemd-oomd** act automatically under pressure, and **zram** provides compressed swap in RAM, but they follow fixed rules and cannot explain or accept a request in natural language.

Aspect	Copilot CLI	Warp AI	Open-source shell LLMs	Proposed agent
Reads live memory state	Not by design	Limited	Only if prompted to run commands	Yes, through MCP tools
Domain focus	General	General	General	Memory and swap
Command safety	User approval	User approval	Often minimal	Allow-list, schema check, confirmation
Runs locally	Cloud model	Cloud service	Depends on setup	Local server; model can be local or hosted
Explains the result	Yes	Yes	Partly	Yes, with measured values

Table 2.2: Comparison of existing approaches with the proposed agent

Research gap. The existing tools show that natural-language command generation works, but none of the ones studied combines (a) live, structured measurements of memory and swap, (b) a narrow set of privileged operations with built-in safety checks, and (c) explanations that tell the user when an action such as dropping the cache will not solve the real problem. This project aims to fill that gap for the memory and swap domain, and the same design can be repeated for the other agents in the series.

References

[1] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[2] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[3] X. V. Lin, C. Wang, L. Zettlemoyer and M. D. Ernst, "NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System," in *Proc. 11th Int. Conf. on Language Resources and Evaluation (LREC)*, 2018.

[4] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Int. Conf. on Learning Representations (ICLR)*, 2023.

[5] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in *NeurIPS*, 2023.

[6] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[7] Anthropic, "Model Context Protocol Specification," modelcontextprotocol.io.

[8] The Linux Kernel documentation, "Documentation for /proc/sys/vm/," kernel.org.

[9] Linux manual pages: proc(5), free(1), vmstat(8), swapon(8), sysctl(8).